

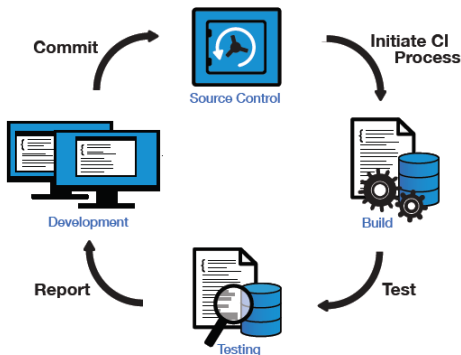
Continuous Integration (SB5-MAI)

Jan Corfixen Sørensen

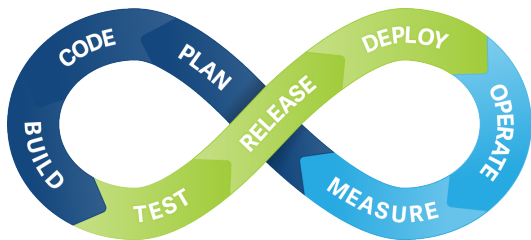
University of Southern Denmark

Continuous Integration

- ▶ Continuous Update for the whole software process
- ▶ Teams integrate their work multiple times per day
- ▶ Each integration is verified by an automated build
- ▶ Significantly reduces integration problems
- ▶ Develop cohesive software more rapidly



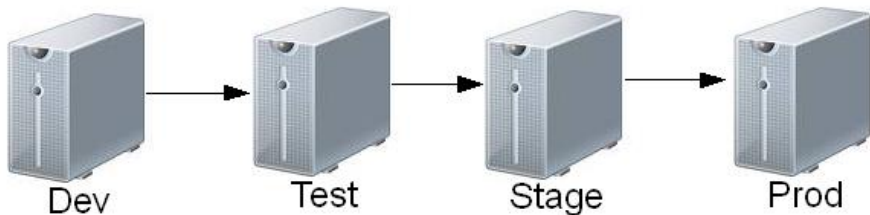
Five Principles of Continuous Integration



- ▶ Environments based on stability
- ▶ Maintain a code repository
- ▶ Commit frequently and build every commit
- ▶ Make the build self-testing
- ▶ Keep the build fast

Continuous Integration

- ▶ Create server environments to model code stability
- ▶ Promote code to stricter environments as quality improves



Commit Frequently Build Every Commit

- ▶ Change your habits
 - ▶ Commit small, functional changes
 - ▶ Unit tests!
 - ▶ Team owns the code, not the individual

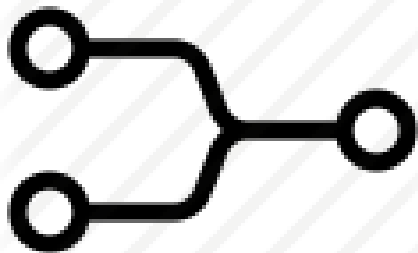


The code builds on my box...

- ▶ Source code repository is the source of record
- ▶ Build server settles disputes
 - ▶ Only gets code from Repo
- ▶ Build server the final authority on stability/quality



Build every commit



- ▶ Why compile frequently?
- ▶ Agile principles
 - ▶ If it hurts, do it more often
 - ▶ Difficult activities can be made more straightforward by doing them more frequently
 - ▶ Reduce time between defect introduction and removal
- ▶ Automate the build

Add testing to build

- ▶ Individual programmers $< 50\%$ efficient at finding their own bugs
- ▶ Multiple quality methods lead to more defects discovered
 - ▶ Use 3 or more methods for $> 90\%$ defect removal
- ▶ Most effective methods
 - ▶ Design inspections
 - ▶ Code inspections
 - ▶ Testing

Self Testing Builds

- ▶ System Tests
 - ▶ End-to-end test
 - ▶ Often take minutes to hours to run
- ▶ Unit tests
 - ▶ **Fast:** No database or file system
 - ▶ **Focused:** Pinpoint problems
 - ▶ Best method for verifying builds



Automated Quality with Continuous Integration



- ▶ Static code analysis
 - ▶ Looks for common java bugs (Findbugs, PMD)
 - ▶ Check for code compliance (Checkstyle)
- ▶ Unit test analysis
 - ▶ Measure coverage (Cobertura)
 - ▶ Look for hotspots, areas of low testing and high complexity (SONAR)

Build Server Hardware



- ▶ Maven and Java = lots of memory
- ▶ Compile and unit test = lots of CPU
- ▶ Static analysis = lots and lots of CPU
- ▶ **Please, KEEP IT FAST**

Test Frameworks



- ▶ Junit
- ▶ Nunit
- ▶ MSTest
- ▶ Selenium
- ▶ Fitnesse

Static Analysis

- ▶ Checkstyle
- ▶ CodeScanner
- ▶ DRY
- ▶ Crap4j
- ▶ Findbugs
- ▶ PMD
- ▶ Fortify
- ▶ Sonar
- ▶ FXCop



Code Coverage

```
public boolean addAll(int index, Collection c) {  
    if(c.isEmpty()) {  
        return false;  
    } else if(_size == index || _size == 0) {  
        return addAll(c);  
    } else {  
        Listable succ = getListableAt(index);  
        Listable pred = (null == succ) ? null : succ.prev();  
        Iterator it = c.iterator();  
        while(it.hasNext()) {  
            pred = insertListable(pred, succ, it.next());  
        }  
        return true;  
    }  
}
```

1 of 2 branches missed.
Press 'F2' for focus

- ▶ Emma
- ▶ Cobertura
- ▶ Clover
- ▶ GCC/GCOV